Singly Linked List – Get Nth Node 2 – Space Complexity and Algorithm

Program :

```
int getNthNode2(Node **head, int node)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return -1;
    }
    if (node < 0)
    {
        return -1;
    }
    else
    {
        Node *current = *head;
        int i = 0;
        while (i < node && current != nullptr)
        {
            current = current->next;
            i++;
        }
        if(current == nullptr){
            return -1;
        }

        else{
            return current->data;
        }
    }
}

…..
case 19:
```

```
        {
            cout << endl;
            cout << endl;
            cout << "Nth node from beginning , where 0 is the first node(0 based
indexing):" << endl;
            int n;
            cout << "Enter the node: ";
            cin >> n;
            if(getNthNode2(&head,n)==-1){
                cout <<"Error:Invalid Index" << endl
            }
            else{
                cout << "The " << n << "th node is from the beginning: " <<
getNthNode2(&head, n) << endl;
            }
            cout << endl;
            cout << endl;
            break;
        }
```

Space Complexity Analysis

1. Input Space Complexity

A. Parameter – Only Definition

The input space complexity refers to the space required for the input parameters of the function.

In this case, the function takes:

- ***A pointer to a pointer (Node ** head), which requires a constant amount of space, $O(1)$.***
- ***An integer (int node), which also requires a constant amount of space, $O(1)$.***

Thus, the total input space complexity is: $O(1)$.

• Node **head – one pointer-to-pointer (machine word)

• int node – one scalar (machine word)

Two fixed-size words → constant memory, independent of the list length.

This view only considers the memory for the parameters passed to the function. The parameters are `Node **head` (a pointer-to-a-pointer) and `int node` (an integer), both of which have a **fixed, constant size**. Therefore, the input space complexity is **O(1)**.

$B. Full - data\ (inclusive) Definition$:

Add everything reachable from the input pointers at the instant the call begins. *head may reference a singly-linked list containing n nodes. Memory for those n nodes dominates, so the total input footprint is Θ(n).

This more common definition includes the entire data structure. If the linked list has **`n` nodes**, the space required to store that list is proportional to `n`. Therefore, the input space complexity is **Θ(n)**.

2. $Auxiliary\ Space\ Complexity$

$Auxiliary\ space\ complexity\ refers\ to\ the\ extra\ space\ used\ by\ the\ algorithm\ that\ is\ not\ part\ of\ the\ input. In\ the\ case\ of\ the\ getNthNode2\ function, the\ auxiliary\ space\ is\ determined\ by\ the\ variables\ declared\ within\ the\ function$:

- $The\ function\ uses\ a\ pointer\ Node\ *current\ to\ traverse\ the\ linked\ list, which\ requires\ O(1) space.$

- $An\ integer\ variable\ int\ i\ is\ used\ to\ keep\ track\ of\ the\ current\ index\ during\ traversal, which\ also\ requires\ O(1)\ space.$

$Since\ both\ of\ these\ variables\ occupy\ a\ constant\ amount\ of\ space\ regardless\ of\ the\ size\ of\ the\ linked\ list, the\ auxiliary\ space\ complexity\ is: O(1)$

What the routine itself adds beyond its input:

• Node *current – one pointer

• int i – one scalar

• A few compiler-generated temporaries / return address

No dynamic heap allocations, no recursion, and the variables are reused throughout the while loop, so the amount of additional memory never grows with n.

Auxiliary space is the extra memory an algorithm uses, not counting the input. For this function:

1. **`Node *current` and `int i`**: The function declares one pointer and one integer to traverse the list and keep count. Both of these variables occupy a constant amount of memory.
2. **Iteration**: The `while` loop reuses these two variables without allocating any new memory.

Since the function only needs a fixed number of extra variables (one pointer and one integer), the amount of extra memory used is constant, regardless of the list's length.

Therefore, the auxiliary space complexity is **$\Theta(1)$**.

$Algorithm$

$GetNthNode2(START, NODE)$

$1.\ [START\ is\ NULL?]\ if\ START = NULL, then:$

$Write: ``\ Error: List\ is\ Empty.``$

$Return\ -1\ and\ exit.$

$[END\ of\ If\ structure]$

$2.\ if\ NODE < 0\ , then:$

$Return\ -1\ and\ Exit.$

$Else$

$SET\ CURRPTR := START\quad [CURRPTR\ will\ point\ to\ Starting\ NODE]$

$SET\ Count := 0$

$Repeat\ while\ Count < NODE\ , AND\ CURRPTR \neq NULL\ then:$

$SET\ CURRPTR := LINK[CURRPTR]$

$SET\ COUNT := COUNT + 1$

$[END\ OF\ Loop]$

$if\ CURRPTR = NULL, then:$

$Return\ -1\ and\ exit.$

$ELSE$

$Return\ INFO[CURRPTR]$

$[END\ of\ If\ structure]$

$[END\ of\ If\ structure]$

$[End\ of\ Algorithm]$